TP 1

Objectifs du TP

- Manipuler vecteurs et matrices avec NumPy (produits, normes, similarité) ;
- résoudre des systèmes linéaires, comprendre déterminant et conditionnement ;
- implémenter la méthode de la puissance et l'appliquer au PageRank ;
- implémenter l'ACP " à la main " et la comparer à `sklearn.decomposition.PCA`.

Consignes

Travail en binôme dans un notebook Jupyter nommé `tp1_nom1_nom2.ipynb`. Imports autorisés : `numpy`, `matplotlib`, `scipy`, `sklearn` (uniquement là où l'énoncé l'indique). Chaque réponse doit être accompagnée d'une phrase d'interprétation. Le notebook doit s'exécuter de bout en bout (*Restart & Run All*) sans erreur.

Exercice 1 — Vecteurs, similarité et " arithmétique des mots "

On fournit le fichier `mini_embeddings.npy` (dictionnaire de 12 mots → vecteurs de $\mathbb{R}^{50}$, extraits de GloVe). À défaut, générez des vecteurs aléatoires pour tester vos fonctions.

1. Écrire une fonction `cosinus(u, v)` qui renvoie $\cos(u,v) = \dfrac{\langle u, v\rangle}{\|u\|_2 \|v\|_2}$ *sans boucle Python* (opérations NumPy vectorisées uniquement).
2. Vérifier numériquement l'inégalité de Cauchy–Schwarz $|\langle u, v\rangle| \leq \|u\|\,\|v\|$ sur 1000 paires de vecteurs aléatoires.
3. Pour chaque mot du dictionnaire, afficher les 3 mots les plus proches au sens du cosinus. Les résultats sont-ils sémantiquement cohérents ?
4. Calculer le vecteur $w = w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}}$ et chercher le mot du dictionnaire le plus proche de w. Commenter.
5. Comparer le classement obtenu avec la distance euclidienne au lieu du cosinus. Dans quel cas les deux classements coïncident-ils ? (*penser aux vecteurs normalisés*).

Exercice 2 — Systèmes linéaires, déterminant et conditionnement

1. Construire la matrice $A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$ et $b = (1, 2, 3)^{\mathsf{T}}$. Calculer $\det A$, puis résoudre $Ax = b$ avec `np.linalg.solve`. Vérifier $Ax \approx b$.
2. Comparer les temps d'exécution de `np.linalg.solve(A, b)` et `np.linalg.inv(A) @ b` pour des matrices aléatoires 500×500 (utiliser `%timeit`). Pourquoi ne faut-il (presque) jamais inverser explicitement une matrice ?

3. Matrice de Hilbert H_n : $H_{ij} = 1/(i+j-1)$. Pour $n = 5, 8, 12$: calculer le conditionnement $\kappa(H_n)$ (`np.linalg.cond`), résoudre $H_n x = b$ avec $b = H_n \mathbf{1}$ (solution exacte $\mathbf{1}$) et mesurer l'erreur $\|x - \mathbf{1}\|_\infty$. Commenter le lien entre conditionnement et perte de précision.

4. En ML, la régression linéaire résout $(X^\top X)\, w = X^\top y$ (équations normales). Expliquer en 3 lignes pourquoi des colonnes de X fortement corrélées rendent ce système mal conditionné, et citer une parade (régularisation ridge : $X^\top X + \lambda I$).

Exercice 3 — Valeurs propres et PageRank

Le graphe du " mini-web " suivant est fourni : 6 pages, liens

$$1\to2,\ 1\to3,\ 2\to4,\ 3\to4,\ 4\to1,\ 4\to5,\ 5\to6,\ 6\to4.$$

1. Construire la matrice de transition P (colonne-stochastique) : $P_{ij} = 1/d_j$ si la page j pointe vers i (d_j : nombre de liens sortants de j).

2. Construire la " matrice de Google " $G = \alpha P + (1-\alpha)\frac{1}{n}\mathbf{1}\mathbf{1}^\top$ avec $\alpha = 0{,}85$. Vérifier que ses colonnes somment à 1.

3. Implémenter la **méthode de la puissance** : partir de $\pi^{(0)} = \mathbf{1}/n$ et itérer $\pi^{(t+1)} = G\pi^{(t)}$ jusqu'à $\|\pi^{(t+1)} - \pi^{(t)}\|_1 < 10^{-10}$. Afficher le classement des pages et le nombre d'itérations.

4. Comparer avec le vecteur propre dominant retourné par `np.linalg.eig`. (Attention : normaliser pour que la somme vaille 1.)

5. Tracer $\|\pi^{(t)} - \pi^\star\|_1$ en fonction de t (échelle log). Quelle est la pente ? La relier au module de la deuxième valeur propre de G.

6. *Bonus* : ajouter une page " piège " (sans lien sortant) et adapter P (colonne uniforme). Que se passe-t-il sans cette correction ?

Exercice 4 — ACP " à la main " sur les chiffres manuscrits

On utilise `sklearn.datasets.load_digits` : $n = 1797$ images 8×8, donc $X \in \mathbb{R}^{1797\times 64}$.

1. Afficher quelques images avec `plt.imshow`. Centrer les données : $X_c = X - \bar{X}$ (moyenne par colonne).

2. Calculer la matrice de covariance $\Sigma = \frac{1}{n-1} X_c^\top X_c$ puis sa décomposition spectrale avec `np.linalg.eigh` (pourquoi `eigh` et pas `eig` ?). Trier valeurs et vecteurs propres par ordre décroissant.

3. Tracer la part de variance expliquée cumulée $k \mapsto \sum_{j\le k} \lambda_j / \sum_j \lambda_j$. Combien de composantes faut-il pour atteindre 90 % ? 99 % ?

4. Projeter les données sur les 2 premières composantes et tracer le nuage de points coloré par le chiffre (0–9). Quels chiffres se séparent bien ?

5. Afficher les 8 premiers vecteurs propres sous forme d'images 8×8 (les " eigen-digits "). Interpréter.

6. Reconstruire les images avec $k \in \{2, 10, 30, 64\}$ composantes : $\widehat{X} = \bar{X} + X_c Q_k Q_k^\top$; afficher côte à côte l'original et les reconstructions pour une même image, avec l'erreur quadratique moyenne.

7. Comparer vos composantes et projections à celles de `sklearn.decomposition.PCA` (`np.allclose`, au signe près — pourquoi le signe peut-il différer ?).

Indication

`np.linalg.eigh` renvoie les valeurs propres en ordre *croissant* : utiliser `[::-1]`. Pour l'affichage des images : `vecteur.reshape(8, 8)`.